

INTERNAL REVIEW DOCUMENT

PIOSI

State of the Project

Printable design bible and production review

This document summarizes the current state of PIOSI for new team members: vision, systems, content architecture, production history, current risks, and review recommendations.

Repository: ZHines2/PIOSI

Source reviewed from: repository checkout

Prepared: 2026-09-13

Executive Summary

PIOSI is a turn-based tactical RPG whose central loop is simple and strong: choose a party of three heroes, survive enemy pressure, break through the wall, and empower the squad between stages.

The repository also shows a second ambition: evolving that playable tactical core into a data-driven content platform built around manifest-loaded hero and level packs.

The project is best understood as a playable combat game with a vivid hero vocabulary, layered with an active but incomplete productization effort. The tactical heart is real and reviewable. The content system is real and promising. Several adjacent systems remain transitional.

Contents

- I. Vision and design philosophy
- II. Core game structure
- III. Battle engine and special mechanics
- IV. Hero roster and stat language
- V. Levels and progression
- VI. Content architecture
- VII. Modes
- VIII. Codebase organization
- IX. Production state and limitations
- X. Production history timeline
- XI. Review guidance
- XII. Citations appendix

I. Vision and Design Philosophy

PIOSI frames battle as myth, endurance, and passage. The Gratt is more than scenery; it is a recurring symbol of resistance, memory, and survival. The heroes are defined less as classes in a formal RPG system than as strong archetypes with compact but expressive stat identities.

Identity through stats

Many hero concepts are expressed through terse properties such as *burn*, *slüj*, *yeet*, *chain*, *psych*, *ankh*, and *rise*.

Progress through breach

The game's decisive objective is often not total elimination but wall destruction and continued advance.

Content over engine churn

The CORE direction aims to let new heroes and levels arrive through content packs instead of repeated engine rewrites.

In short: PIOSI is a tactical wall-breaker with a lore-forward tone and a growing content-platform ambition.

II. Core Game Structure

The main runtime flow is title screen, party selection, battle, mode-up selection, next level, then victory, defeat, or optional mode transitions. The player must choose exactly three heroes before the run begins.

- Party members are cloned from the master roster.
- Turn order is initially sorted by agility descending.
- Each active hero receives movement points equal to agility.
- Enemy turns occur after the party cycle completes.
- Completing a level typically opens Mode Up.

This loop is the most stable and product-defining part of the project.

III. Battle Engine and Special Mechanics

The battle engine builds a grid battlefield, places heroes and enemies, lays a wall row, and spawns one vittle plus one mushroom. Combat is directional and spatial: movement is orthogonal, attacks are line-based, and enemy AI is greedy and adjacency-driven.

- **Heal:** ally-target healing
- **Psych:** random ally stat boost
- **Trick:** random enemy stat reduction
- **Burn:** three-turn damage over time
- **Slüj:** stacking timed damage
- **Yeet:** knockback with collision damage
- **Chain:** diminishing splash propagation
- **Swarm:** adjacent aura damage
- **Spicy:** stronger vittle healing
- **Armor:** charge-based hit negation
- **Spore:** mushroom-based random growth
- **Caprice, Fate, Rage, Ankh, Rise, Dodge, Bomba**

These mechanics form the game's actual design language: terse names, high recognizability, and high variance from a small set of rules.

IV. Hero Roster and Stat Language

The current CORE roster contains twenty-five heroes and is mirrored in a static fallback roster. The cast spans foundational melee and ranged units, support specialists, random-effect units, death-linked units, and experimental edge-case designs.

Representative anchors include Knight, Archer, Wizard, Cleric, Berserker, Jester, Meatwalker, Soothscribe, Nonsequiteur, Griot, Torcher, Sløjier, Shrink, Yeetrian, Mellitron, Gastronomer, Palisade, Mycelian, Pæg, Kemetic, Greenjay, Sysiphuge, and Bombador.

Pæg is especially revealing: it acts almost like a compact catalog of the project's broader stat vocabulary.

Bulk and Ghïs

Bulk is part of the intended combat language. Meatwalker carries the stat in roster data, Mode Up can increase it, and March 2025 production history explicitly records a fix whose stated purpose was to make bulk raise a random stat when an enemy is defeated. The current branch now restores that behavior through the formal `onKill` lifecycle hook in `battleRules.js`, so the code and the documentation agree again.

Ghïs appears mainly as a fallback progression stat and default mode-up behavior for heroes without a bespoke progression rule. It remains part of the stat surface, but there is still no dedicated battle hook for it in the current runtime.

V. Levels and Progression

The static campaign defines levels 1 through 20 plus a special level 99. The currently surfaced CORE content pack exposes levels 1 through 3. This means the repository contains more authored battle content than the active manifest currently serves by default.

Difficulty and atmosphere scale through larger battlefields, stronger enemy formations, greater wall health, more elaborate stage titles, and occasional procedural generation. The later campaign shifts from direct military conflict toward more surreal or symbolic encounters.

VI. Content Architecture

The CORE build loads a manifest, then fetches hero and level JSON packs, merges them, and falls back to static built-in content if loading fails. This is the clearest sign of long-term architectural intent.

- Manifest declares version, pack list, and mode flags.
- Heroes load from `content/heroes.<pack>.json`.
- Levels load from `content/levels.<pack>.json`.
- Fallback content remains available for no-server operation.

The abstraction is meaningful, but not total: adding a new hero still requires a corresponding Mode Up case in code.

VII. Modes

Battle is the primary and most complete mode.

Mode Up is a between-level party empowerment system in which one living hero determines the squad-wide buff profile.

World Map is a hardcoded branching shell labeled “THE BROADLANDS,” with some future-facing placeholder logic still present.

Summit Mode is an auto-resolving battle royale simulation on a large map. It is imaginative but presently diverges from the player manual’s description.

Emanations is a lightweight music-player mode that extends the project’s atmosphere more than its tactics.

VIII. Codebase Organization

- **index.html**: runtime entry, UI flow, input, audio, orchestration
- **battleEngine.js**: combat engine
- **battleRules.js**: lifecycle-based ability rules and kill/death hooks
- **gameModel.js**: normalized state, stable IDs, RNG, and battle phases
- **heroes.js** / **levels.js**: static fallback content
- **contentLoader.js**: manifest and JSON pack loading
- **contentSchema.js**: content normalization and validation
- **modeup.js**: progression rules
- **worldMap.js**, **summitMode.js**, **emanations.js**: side modes
- **griot.js**: flavor APIs and Markov text logic

The code is still centralized in orchestration, but this branch now adds a formal rules layer, deterministic RNG, battle phases, and load-time content normalization to reduce drift between intent, data, and runtime.

IX. Production State and Limitations

Strong, stable areas

- The 3-hero tactical loop
- Mode Up progression
- The broad hero-stat vocabulary
- Manifest-based content loading
- Recent HUD, isometric view, and mobile D-pad polish

Known limitations

- Enemy AI attacks adjacency even when enemy range stats are higher.
- Level layout plumbing is present but not fully coherent end-to-end.
- `coreLevels` is documented but not clearly used at runtime.
- Some manifest flags are more declarative than enforced.
- Summit documentation and implementation do not fully match.
- The build file still references Parcel paths not present in the repo.

X. Production History Timeline

- **2025-01-30** — Initial commit establishes the repository.
- **2025-02-07** — Cleric and healing enter the roster and UI.
- **2025-02-10** — Sycophant is added.
- **2025-02-23** — Chain stat work lands.
- **2025-02-25** — Rage support and later-campaign levels expand the core game.
- **2025-02-26** — Trick and Bulk are added in the roster/progression era.
- **2025-02-27** — Psych and supporting battle changes arrive.
- **2025-02-28** — Level 99 access is corrected.
- **2025-03-04** — Kemetic and Ankh mechanics enter production.
- **2025-03-05** — Greenjay/Rise and Sysiphuge/Dodge are integrated.
- **2025-03-06** — Bulk-on-defeat behavior is explicitly fixed in history.
- **2025-03-08** — Bombador and Bomba are added.
- **2025-03-09** — Summit Mode is introduced.
- **2025-03-13** — Emanations Mode is introduced.
- **2025-03-05** — Documentation is refreshed to reflect expanded content.
- **2026-03-11** — CORE data-driven content loading arrives.
- **2026-02 to 2026-03** — Isometric view, mobile D-pad, and battle HUD polish the experience.

XI. Review Guidance

Preserve

- The tactical 3-hero wall-breaker loop
- The distinctive stat vocabulary
- Mode Up as identity and progression
- The content-pack direction

Verify carefully

- Gh's meaning and future role
- World Map progression relevance
- Summit Mode's intended product status
- Layout support and manifest flag enforcement

Potential cleanup targets

- Dead or disconnected Summit hero-selection path
- Stale Parcel build references
- Legacy archival artifacts if no longer authoritative

PIOSI's next challenge is less invention than consolidation. The project already has a strong center; the work ahead is to make its edges equally coherent.

XII. Citations Appendix

Project overview and CORE direction

README.md:4-13, 71-106

Manifest and content loading

content/manifest.json:1-12

contentLoader.js:25-33, 35-59, 124-153

contentSchema.js:1-108

Main runtime loop and screen flow

index.html:15-33, 378-407, 459-520, 840-994

Battle engine and status systems

battleEngine.js:31-85, 115-132, 168-191, 227-347, 379-565

battleRules.js:8-271

gameModel.js:1-120

Knockback and slüj modules

applyKnockback.js:1-45

sluj.js:1-65

Hero roster

heroes.js:9-286

content/heroes.core.json:2-291

Levels and progression content

levels.js:70-546

content/levels.core.json:2-87

Player docs and lore

docs/players-manual.md:6-25, 149-204

docs/hero-manifestation-guide.md:18-36, 271-318

fantasy_narrative.txt:17-35

World Map, Summit, and Emanations

worldMap.js:1-72

summitMode.js:1-266

emanations.js:1-146

Build and presentation layers

package.json:1-17

styles.css:1-520

Production history references

Commits reviewed include:

97e3c41d059a013617eb22b30c5d703f98da7724, 3e5bbb82a5a694a175cd0de23ad0165e1ec56644, 6e188e9847cbc90535afd3aa3f6737942e659fbb, 2cb96849e5bd69b47a9424fe847666c4e24beec6, a9c7a4048cddb46755c3402b23831f2b9005ba7b, 57dfdc5b57ff0e4a308aab4a4e2ed3c9f96d705e, 53fe7bcbf3dc963e341fddf4b48a4b04460b6c5f, fd963c16111d696ccf36424d6c9f36d7ae4e1cc9, e424fa70d126962d2ea6467f196b55181646d69f, 77025f8f99748c95fc6ebf5537c16a291c7c2c30, ea18069399f63a1ddb410ef6142a60af1061c1e, 724201198bcd40a3395d2c30a6a01deaac94583a, 650fc6a39c5be0b602fa12d3b480ba040b5581be, 23dd3e0c7095cbf3e65cd0b1bb5a9392cb4d3deb, 91c30fd12673fe59826cb1b587bdb0ffc44b35d, 538b2664010c19a687a21f1e54f2fe9ec34f11c3, 20f11bcb212448c6f3d08e0c56ecdb676d95dff, 60fa7b0a4ef57cbffa78c52068d796f2eb84ac0f, 581ee59dd413dddc7bfc1ff2290ca695bdf9b56a, df71ed9125302cbb90ef2f6d71ca8a9b3ff7d31d

Prepared from repository source and commit history review only.